SECURITY ASSESSMENT & PROPOSAL

Signal Chat Relay Container Review

11 October 2026 Subject: the public open-source image `bbernhard/signal-cli-rest-api`, release 0.101

The Signal Chat Relay carries a Signal group chat, one way, to another platform. At its start sits the Signal container: the one part that connects to Signal and holds the account's encryption keys, so it decides how far the whole relay can be trusted. This review assesses the public open-source image available for that role (Part 1) and proposes an in-house Signal Chat Relay built only from verified official Signal releases, with an encrypted, read-only link to the rest of the relay (Part 2).

BOTTOM LINE

Why: the public image contains no evidence of malicious code, and its Signal encryption and protocol components match the official releases. But whatever runs it holds the Signal account's keys, and the image falls short of what a system holding credentials needs:

- Its **control interface has no login**, and the project's own examples make it reachable from the network. Anyone who reaches it can read the account's messages, send as the account, or take the account over.
- It runs **code from a single outside developer** with full access to the account keys, and we can't prove the shipped program matches the published source.
- Its **downloads aren't verified** when it's built, and its advertised **"latest" version changes silently**, so a faulty or tampered release could arrive unnoticed.
- Its link to the rest of the relay is **unencrypted**, which rules out running the parts on separate networks.

Proposal: build an **in-house Signal Chat Relay** from verified official Signal releases only. It has no outside developer's code and no control interface, it checks every component on every build, and it connects to the relay over an encrypted, read-only link that only our own systems can use. Estimated 4–6 working days of one engineer to build, test and deploy; no licence cost.

PART 1

Assessment of the public image

Whatever runs this image holds the Signal account's encryption keys: it can read the account's chats and act as the account. It needs the same scrutiny as any system that holds credentials. We examined release 0.101, the published build recipe, the source code and the project's own deployment examples.

AUTHENTIC

The Signal encryption library and the Signal protocol software match the official releases exactly.

WELL BEHAVED IN TESTING

Running in a test environment, it connected only to Signal's own servers.

OPEN BY DESIGN

Its control interface has no login, and the project's examples make it reachable from the network.

LOOSELY CONTROLLED

Downloads aren't verified when it's built, the advertised version changes silently, and part of the code comes from one maintainer.

What's inside, and what we verified

COMPONENT	PUBLISHED BY	WHAT IT DOES	VERIFIED
Encryption library (libsignal)	Signal Messenger	All encryption and key handling	Identical to Signal's official release, checked on the copy the image actually loads when it runs
Signal protocol software (signal-cli)	signal-cli project	Connects to Signal, sends and receives	Identical: 77 of 79 libraries byte-for-byte; the other 2 differ only by repackaging and the extra library below
Web control interface (REST API)	One outside developer	Wraps signal-cli in a web API; ~14,000 lines	Source reviewed for network calls: only local ones found. We can't prove the shipped program was built from that source.
Extra copy of the encryption library	Same outside developer	Alternative build for other CPU types	Present, not used on standard servers. Confirmed it is not loaded.
Operating system and process supervisor	Debian, s6-overlay	Base system	Widely used; not reviewed

Findings

SEVERITY	FINDING	WHY IT MATTERS	MITIGATION
HIGH	The control interface has no login of any kind, and the project's examples publish it to the network. The documentation gives no warning about this.	Anyone who can reach it can link their own phone to the account and read every message from then on, send messages as the account, or delete the account.	Never expose the interface beyond its own host. The in-house Signal Chat Relay has no such interface.
MEDIUM	The project's examples use the moving "latest" version rather than a fixed one	A routine update silently installs whatever was last published, including a faulty or compromised release.	Pin an exact, reviewed version. The in-house Signal Chat Relay only changes when we change it.
MEDIUM	The image's build recipe downloads its components without checking fingerprints or signatures	If a download source were tampered with, the change would flow into the published image unnoticed.	The in-house Signal Chat Relay checks every download against a recorded fingerprint.
MEDIUM	A single outside maintainer's code runs with full access to the account keys	Users depend on one person's security practices, and can't prove the shipped program matches the reviewed source.	The in-house Signal Chat Relay doesn't include this code.
LOW	The image carries far more than a one-way relay needs: sending, account management, device linking, add-on scripts that can make web requests, and outgoing notifications	Every unused feature is a way the account could be misused if something else went wrong.	The in-house Signal Chat Relay contains only what the relay uses.
INFO	An unused extra copy of the encryption library, built by the maintainer, is in the image	No effect on standard servers; it's unneeded code from a non-official source.	Not included in the in-house Signal Chat Relay.

On the positive side, the image already runs as a regular user rather than an administrator, and its recipe supports a read-only system. No software, open or closed source, can be proven free of malicious code. Assurance comes from three things together: verifying that what runs

matches trusted releases, limiting what it could do if it misbehaved, and watching for unexpected behaviour. The proposal below is built around all three.

PART 2

Proposal: an in-house Signal Chat Relay

The official Signal protocol software already includes its own built-in service for streaming incoming messages. We confirmed this in its recent releases. That makes the outside developer's web interface unnecessary for a one-way relay. We'd build a small image of our own that contains only the official Signal components, verified on every build, and nothing else.

Who we'd have to trust

Public image		In-house Signal Chat Relay	
Container image	outside developer	Container image	us: build file + secure gateway
Web control interface	outside developer	Signal protocol software	signal-cli project, verified
Signal protocol software	signal-cli project	Encryption library	Signal Messenger, verified
Encryption library	Signal Messenger		

outside, single maintainer · official release, fingerprint-checked · built and controlled by us

How it fits the relay

```
Signal servers
     |  encrypted, outbound only
     ▼
[ Signal container ]   in-house Signal Chat Relay · official Signal software only
     |  encrypted link (TLS 1.3, both sides prove identity) · same host or across networks
     ▼
[ Subscriber ] → CEF over UDP → data diode → [ Receiver ] → local chat playback (POC), external platform later on
```

Each part has one job. The Signal container is the only part that holds the account's keys or talks to Signal. The subscriber reads incoming messages from it over the encrypted link, keeps only the chat being relayed, and converts each message to CEF. From the subscriber onward, only CEF over UDP travels, in one direction, so nothing downstream ever reaches back to the Signal container or the account.

Security controls built in

Every control below applies to the Signal container alone. Nothing changes on the shared host's firewall or for the other applications running there.

- **Verified inputs:** every component is checked against its official published fingerprint on every build. A mismatch stops the build.
- **Pinned versions:** nothing changes unless we change it, and each change is reviewed.
- **No exposed control interface:** only the subscriber, holding its own certificate, can connect. The container offers no web pages, no login screens and no device-linking over the network.

- **Encrypted, authenticated link:** the connection to the subscriber is encrypted and both ends prove who they are (details below).
- **Locked-down container:** runs as a regular user, with a read-only system, no extra privileges, and memory and process limits.
- **Linked-device check:** a regular check of the devices linked to the account, with an alert on any new one.
- **Vulnerability scan** of the image on every build, for known published flaws.

Encrypted link between the Signal container and the subscriber

The Signal container and the subscriber can run on the same host or on separate hosts in different networks. Because message text travels over this link, it is always encrypted, in both layouts, and the image adds a small secure gateway in front of the Signal software's built-in message service. The gateway uses only standard, built-in Java components, so no outside code is involved.

- **Encryption:** TLS 1.3 only, the current standard; older versions are refused.
- **Both sides prove their identity:** the subscriber must present its own certificate, and it accepts only our Signal container's certificate. Connections without a valid certificate are refused before any data is exchanged.
- **Our own certificates:** issued by a private certificate authority we control, not a public one. Each host receives only its own key. The authority's signing key is kept offline and used only to issue or renew certificates.
- **Read-only by design:** the gateway allows only reading incoming messages and looking up display names. Sending messages and every other account action are blocked, even for a connection with a valid certificate. This enforces the relay's one-way rule in the Signal container itself.
- **Never falls back to unencrypted:** if the certificates are missing or invalid, the Signal container refuses to start. The unencrypted built-in service stays inside the container and is never reachable from the network.
- **Simple to place:** moving between one host and two is a configuration change, not a code change. The network port is a single setting (443 by default).
- **No lost messages on reconnect or move:** the subscriber records how far it has read, and the Signal container resends anything missed.

When the two run on separate hosts, the network must allow the subscriber's host to connect to the Signal container's host on the chosen port. This is the only network requirement, and it is a standard firewall request between the two hosts, not a change to the shared host.

Plan and effort

Build	**Build the in-house Signal Chat Relay** Build file with fingerprint checks, the secure gateway and certificates, vulnerability scan, written build and update procedure.	2–3 days
Test	**Test the Signal Chat Relay** Confirm it connects to Signal, receives and streams messages correctly, and handles reconnects without losing messages. Confirm the gateway refuses connections without a valid certificate and blocks sending. Confirm admin tasks (linking a device, registering a number) work.	1–2 days

Deploy	**Deploy** Issue the certificates, install the image with its locked-down settings, and connect the subscriber over the encrypted link.	½–1 day

Estimates assume one engineer and include testing. Total about 4–6 working days.

Trade-offs and risks

TRADE-OFF	IMPACT	HOW WE HANDLE IT
We own updates	Signal stops serving clients that fall too far behind, so the Signal software must be kept current. With the public image, the outside developer does this.	A written monthly update and verification routine, delivered with the image. Each update is a version number and fingerprint change.
No web admin screens	Occasional admin tasks, such as linking a device, are done from the server's command line.	For a one-way relay this is a benefit: there are fewer ways to act on the account.
The built-in message service has no login	Anything that can reach it could use it.	It stays inside the container. The only way in is the secure gateway, which requires the subscriber's certificate and allows reading only.
Small change to the subscriber	New connection code, plus a new way to look up display names.	Covered in testing.
Certificates to look after	The link's certificates must be renewed before they expire.	A script issues and renews them; renewal is a short, scheduled task.
Open-source licences	The Signal components are GPL and AGPL licensed.	Internal use is permitted. Obligations apply only if a modified version is given outside the organisation; we don't modify them.

Options considered

	USE THE PUBLIC IMAGE	CUSTOMISE THE PUBLIC IMAGE	BUILD AN IN-HOUSE SIGNAL CHAT RELAY (RECOMMENDED)
What it means	Deploy it as published, with careful settings	Take a private copy of the outside code, add a login and remove unneeded features	Our own small image with only the official Signal components
Outside code with key access	Yes	Yes, but ours to review	None
Control interface	Yes, no login	Yes, with a login	None
Link to the subscriber	Unencrypted, no login	Depends on the changes made	Encrypted; certificate required; read-only
Code we must maintain	None	~14,000 lines, merged with every upstream change	A short build file and a small gateway, plus small subscriber changes
Effort	—	2–3 weeks	4–6 days

Evidence

- The encryption library the image loads at run time matches Signal's official libsignal 0.102.1 release (SHA-256 `00ecda2a…6662d`).

- signal-cli 0.14.8 libraries compared one by one with the official GitHub release: 77 of 79 byte-identical; one repackaged with identical contents; one carrying the unused extra library.
- Source of the web interface reviewed at the exact release examined (0.101), including every outbound network call and its full list of control endpoints. No login check exists anywhere in the code.
- The project's README and example deployment file were reviewed: both use the "latest" version and publish the control interface, with no security warning.
- The upstream build recipe was reviewed for download verification and version pinning.
- In a test environment, the running image connected only to Signal's own servers.
- The built-in message service, with a live event stream, was confirmed present in signal-cli 0.14.8.